Assignment-16.3

Task 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System

Prompt:-

Create a normalized SQL schema for a Hospital Management System using SQLite with tables Doctors, Patients, and Appointments.

Include primary keys, foreign keys, NOT NULL, UNIQUE, and CHECK constraints (appointment_date should be valid using datetime('now')).

Insert sample data.

Write SQL queries using JOINs to:

1. List all appointments for a given doctor_id
2. Retrieve full patient history using patient_id
3. Count total patients treated by each doctor using GROUP BY

eviewed and created hospital.sql

Code:-

hospital.sql

```sql
-- Create Doctors table
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    specialty TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
-- Create Patients table
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    dob DATE NOT NULL,
    phone TEXT UNIQUE NOT NULL
);
-- Create Appointments table
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date DATETIME NOT NULL CHECK(appointment_date >= datetime('now')),
    reason TEXT,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
-- Insert sample data into Doctors
INSERT INTO Doctors (name, specialty, email) VALUES
('Dr. John Smith', 'Cardiology', 'john.smith@hospital.com'),
('Dr. Emily Johnson', 'Neurology', 'emily.johnson@hospital.com'),
('Dr. Michael Brown', 'Orthopedics', 'michael.brown@hospital.com');
-- Insert sample data into Patients
INSERT INTO Patients (name, dob, phone) VALUES
('Alice Wilson', '1985-05-15', '555-0101'),
('Bob Davis', '1990-12-20', '555-0102'),
('Charlie Miller', '1978-03-10', '555-0103'),
('Diana Garcia', '1982-07-25', '555-0104');
-- Insert sample data into Appointments
INSERT INTO Appointments (doctor_id, patient_id, appointment_date, reason) VALUES
(1, 1, '2026-04-05 10:00:00', 'Routine checkup'),
(1, 2, '2026-04-06 11:00:00', 'Follow-up'),
(2, 3, '2026-04-07 14:00:00', 'Consultation'),
(2, 4, '2026-04-08 15:00:00', 'Initial visit'),
(3, 1, '2026-04-09 09:00:00', 'Surgery consultation'),
(3, 2, '2026-04-10 16:00:00', 'Physical therapy');
-- Query 1: List all appointments for a given doctor_id (using JOIN to include patient info)
-- Replace ? with actual doctor_id, e.g., 1
SELECT a.appointment_id, a.appointment_date, a.reason, p.name AS patient_name, p.phone
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = ?;
-- Query 2: Retrieve full patient history using patient_id (using JOIN to include doctor info)
-- Replace ? with actual patient_id, e.g., 1
SELECT a.appointment_id, a.appointment_date, a.reason, d.name AS doctor_name, d.specialty
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.patient_id = ?;
-- Query 3: Count total patients treated by each doctor using GROUP BY
SELECT d.name AS doctor_name, COUNT(DISTINCT a.patient_id) AS total_patients
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.name;
```

Output:-

SQL ▾ 1 / 1 1 - 0 of

SQL ▾ 1 / 1 1 - 3 of

doctor_name	total_patients
Dr. John Smith	2
Dr. Emily Johnson	2
Dr. Michael Brown	2

Task 2 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform

Prompt:-

Create a SQLite database for an E-commerce system with tables Users, Products, Orders, and OrderDetails.

Add primary keys, foreign keys, NOT NULL, UNIQUE, and CHECK constraints.

Insert sample data.

Write SQL queries to:

Get all orders for a user_id
Find the most purchased product
Calculate total monthly revenue using JOINs and GROUP BY

Code:-

ecommerce.sql

```sql
-- RESET (avoid table already exists error)
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;
-- Users table
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    address TEXT NOT NULL
);
-- Products table
CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    description TEXT,
    price REAL NOT NULL CHECK(price > 0),
    stock_quantity INTEGER NOT NULL CHECK(stock_quantity >= 0)
);
-- Orders table
CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    order_date DATETIME NOT NULL,
    total_amount REAL NOT NULL CHECK(total_amount >= 0),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
-- OrderDetails table
CREATE TABLE OrderDetails (
    order_detail_id INTEGER PRIMARY KEY,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL CHECK(quantity > 0),
    price_at_time REAL NOT NULL CHECK(price_at_time > 0),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
-- Sample data
INSERT INTO Users (name, email, address) VALUES
('Alice Johnson', 'alice@example.com', '123 Maple St'),
('Bob Smith', 'bob@example.com', '456 Oak Ave'),
('Carol Davis', 'carol@example.com', '789 Pine Rd');
INSERT INTO Products (name, description, price, stock_quantity) VALUES
('Smartphone', 'Latest model smartphone', 699.99, 100),
('Laptop', 'High-performance laptop', 1299.99, 50),
('Headphones', 'Noise-cancelling headphones', 199.99, 200),
('Tablet', '10-inch tablet', 399.99, 75);
INSERT INTO Orders (user_id, order_date, total_amount) VALUES
(1, '2026-01-15 10:00:00', 899.98),
(1, '2026-02-20 14:30:00', 1499.98),
(2, '2026-02-25 16:00:00', 699.99),
(2, '2026-03-10 11:15:00', 399.99),
(3, '2026-03-15 09:45:00', 1699.98),
(3, '2026-04-01 13:20:00', 699.99);
```

```sql
(3, '2026-03-15 09:45:00', 1699.98),
(3, '2026-04-01 13:20:00', 699.99);
INSERT INTO OrderDetails (order_id, product_id, quantity, price_at_time) VALUES
(1, 1, 1, 699.99),
(1, 3, 1, 199.99),
(2, 2, 1, 1299.99),
(2, 3, 1, 199.99),
(3, 1, 1, 699.99),
(4, 4, 1, 399.99),
(5, 2, 1, 1299.99),
(5, 4, 1, 399.99),
(6, 1, 1, 699.99);
-- 1. Retrieve all orders by a user
SELECT u.name, o.order_id, o.order_date, o.total_amount
FROM Orders o
JOIN Users u ON o.user_id = u.user_id
WHERE o.user_id = 1;
-- 2. Most purchased product
SELECT p.name, SUM(od.quantity) AS total_quantity
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_id
ORDER BY total_quantity DESC
LIMIT 1;
-- 3. Monthly revenue
SELECT strftime('%Y-%m', o.order_date) AS month,
       SUM(od.quantity * od.price_at_time) AS revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
GROUP BY month
ORDER BY month;

-- Indexes (optimization)
CREATE INDEX idx_user_id ON Orders(user_id);
CREATE INDEX idx_product_id ON OrderDetails(product_id);
```

Output:-

SQL ▼ 1 / 1 1 - 0 of 0

SQL ▼ 1 / 1 1 - 2 of 2

name	order_id	order_date	total_amount
Alice Johnson	1	2026-01-15 10:00:00	899.98
Alice Johnson	2	2026-02-20 14:30:00	1499.98

SQL ▼ 1 / 1 1 - 1 of 1

name	total_quantity
Smartphone	3

SQL ▼ 1 / 1 1 - 4 of 4

month	revenue
2026-01	899.98
2026-02	2199.97
2026-03	2099.97
2026-04	699.99

SQL ▼ 1 / 1 1 - 0 of 0

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Prompt:-

Create a SQLite database for a Library Management System with tables Books, Members, and Loans.
Include primary keys, foreign keys, NOT NULL, UNIQUE, and CHECK constraints.
Insert sample data.
Write SQL queries using JOINs to:

1. Retrieve all books currently issued (return_date IS NULL)
2. Find overdue books (loan_date older than 30 days)
3. Count number of books loaned by each member using GROUP BY
   Also suggest indexing strategies for faster queries.

Code:-

library.sql

```sql
-- Library Management System Database Schema
-- Create Books table
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    isbn TEXT UNIQUE NOT NULL,
    publication_year INTEGER CHECK(publication_year > 0),
    available_copies INTEGER NOT NULL CHECK(available_copies >= 0)
);
-- Create Members table
CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    phone TEXT,
    join_date DATE NOT NULL
);
-- Create Loans table
CREATE TABLE Loans (
    loan_id INTEGER PRIMARY KEY,
    book_id INTEGER NOT NULL,
    member_id INTEGER NOT NULL,
    loan_date DATE NOT NULL,
    due_date DATE NOT NULL CHECK(due_date > loan_date),
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
-- Insert sample data into Books
INSERT INTO Books (title, author, isbn, publication_year, available_copies) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', '978-0-7432-7356-5', 1925, 5),
('To Kill a Mockingbird', 'Harper Lee', '978-0-06-112008-4', 1960, 3),
('1984', 'George Orwell', '978-0-452-28423-4', 1949, 4),
('Pride and Prejudice', 'Jane Austen', '978-0-14-143951-8', 1813, 6),
('The Catcher in the Rye', 'J.D. Salinger', '978-0-316-769488-0', 1951, 2);
-- Insert sample data into Members
INSERT INTO Members (name, email, phone, join_date) VALUES
('John Doe', 'john.doe@example.com', '555-1234', '2020-01-15'),
('Jane Smith', 'jane.smith@example.com', '555-5678', '2019-05-20'),
('Bob Johnson', 'bob.johnson@example.com', '555-9012', '2021-03-10'),
('Alice Brown', 'alice.brown@example.com', '555-3456', '2022-07-25');
-- Insert sample data into Loans
INSERT INTO Loans (book_id, member_id, loan_date, due_date, return_date) VALUES
(1, 1, '2026-03-01', '2026-03-31', NULL),
(2, 2, '2026-03-15', '2026-03-15', '2026-03-10'),
(3, 3, '2026-03-05', '2026-04-05', NULL),
(4, 3, '2026-01-20', '2026-02-20', '2026-02-18'),
(5, 4, '2026-03-10', '2026-04-10', NULL),
(1, 2, '2026-02-01', '2026-03-01', '2026-02-28'),
(3, 3, '2026-02-20', '2026-03-20', NULL);
-- Query 1: Retrieve all books currently issued (return_date IS NULL)
SELECT b.title, b.author, m.name AS borrower_name, l.loan_date, l.due_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
-- Query 2: Find overdue books (loan_date older than 30 days)
SELECT b.title, b.author, m.name AS borrower_name, l.loan_date, l.due_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL AND julianday('now') - julianday(l.loan_date) > 30;
-- Query 3: Count number of books loaned by each member using GROUP BY
SELECT m.name AS member_name, COUNT(l.loan_id) AS books_loaned
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_id, m.name
ORDER BY books_loaned DESC;
-- Indexing strategies for faster queries:
-- CREATE INDEX idx_loans_book_id ON Loans(book_id);
-- CREATE INDEX idx_loans_member_id ON Loans(member_id);
-- CREATE INDEX idx_loans_return_date ON Loans(return_date);
-- CREATE INDEX idx_loans_loan_date ON Loans(loan_date);
-- CREATE INDEX idx_books_title ON Books(title);
-- CREATE INDEX idx_members_name ON Members(name);
-- These indexes will speed up JOINs, WHERE clauses on return_date and loan_date, and searches by title or member name.
```

Output:-

SQL ▾ 1 / 1 1 - 4 of 4

title	author	borrower_name	loan_date	due_date
The Great Gatsby	F. Scott Fitzgerald	John Doe	2026-03-01	2026-03-31
1984	George Orwell	John Doe	2026-03-05	2026-04-05
The Catcher in the Rye	J.D. Salinger	Alice Brown	2026-03-10	2026-04-10
1984	George Orwell	Bob Johnson	2026-02-20	2026-03-20

SQL ▾ 1 / 1 1 - 2 of 2

title	author	borrower_name	loan_date	due_date
The Great Gatsby	F. Scott Fitzgerald	John Doe	2026-03-01	2026-03-31
1984	George Orwell	Bob Johnson	2026-02-20	2026-03-20

SQL ▾ 1 / 1 1 - 4 of 4

member_name	books_loaned
John Doe	2
Jane Smith	2
Bob Johnson	2
Alice Brown	1

Task 4: Optimize Queries

Instructions:

- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Prompt:-

Analyze the given SQL queries for efficiency and optimize them by replacing subqueries with JOINs, adding indexes, and simplifying conditions. Ensure the optimized queries return the same results and explain why they perform better.

Queries.sql

```sql
-- Original Query: Find books by authors from the UK using subquery
SELECT *
FROM Books
WHERE author_id IN (
    SELECT author_id
    FROM Authors
    WHERE country='UK'
);

-- Optimized Query: Use JOIN instead of subquery
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';

-- Explanation:
-- The original query uses a subquery with IN, which may execute the subquery for each row in Books or use a semi-join.
-- The optimized version uses an INNER JOIN, which is typically faster because:
-- - It allows the query optimizer to choose efficient join algorithms (e.g., hash join or merge join).
-- - Reduces the need for nested execution.
-- - With proper indexes, it can perform index lookups instead of table scans.
-- Both queries return the same results: all books where the author is from the UK.

-- Indexing Recommendations:
-- CREATE INDEX idx_authors_country ON Authors(country);
-- CREATE INDEX idx_books_author_id ON Books(author_id);
-- These indexes will speed up the WHERE clause on country and the JOIN on author_id, making the optimized query even faster.
```

Task 5: University Course Registration

Scenario:

SR University needs a course registration system for students enrolling in different courses under faculty supervision

Prompt:-

Create a SQLite database schema for a University Course Registration system with tables Students, Courses, Faculty, and Registrations.
Define relationships:

- Each student can register for multiple courses
- Each course is taught by one faculty member

Include primary keys, foreign keys, NOT NULL, and UNIQUE constraints.
Insert sample data.
Write SQL queries using JOINs to:

1. List all students enrolled in a specific course
2. Find faculty members teaching more than 2 courses
3. Retrieve courses with the highest number of registrations using GROUP BY

Ensure proper normalization and query optimization.

Code:-

University.sql

```sql
-- University Course Registration System Database Schema
-- Create Students table
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    major TEXT NOT NULL,
    enrollment_year INTEGER NOT NULL
);
-- Create Faculty table
CREATE TABLE Faculty (
    faculty_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    department TEXT NOT NULL
);
-- Create Courses table
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    code TEXT UNIQUE NOT NULL,
    credits INTEGER NOT NULL CHECK(credits > 0),
    faculty_id INTEGER NOT NULL,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);
-- Create Registrations table
CREATE TABLE Registrations (
    registration_id INTEGER PRIMARY KEY,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    registration_date DATE NOT NULL,
    grade TEXT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id)  -- Prevent duplicate registrations
);
-- Insert sample data into Students
INSERT INTO Students (name, email, major, enrollment_year) VALUES
('Alice Johnson', 'alice.johnson@university.edu', 'Computer Science', 2022),
('Bob Smith', 'bob.smith@university.edu', 'Mathematics', 2021),
('Carol Davis', 'carol.davis@university.edu', 'Physics', 2023),
('David Wilson', 'david.wilson@university.edu', 'Chemistry', 2020),
('Eva Brown', 'eva.brown@university.edu', 'Biology', 2022);
-- Insert sample data into Faculty
INSERT INTO Faculty (name, email, department) VALUES
```

```sql
INSERT INTO Faculty (name, email, department) VALUES
('Dr. John Doe', 'john.doe@university.edu', 'Computer Science'),
('Dr. Jane Smith', 'jane.smith@university.edu', 'Mathematics'),
('Dr. Michael Johnson', 'michael.johnson@university.edu', 'Physics'),
('Dr. Sarah Davis', 'sarah.davis@university.edu', 'Chemistry');
-- Insert sample data into Courses
INSERT INTO Courses (title, code, credits, faculty_id) VALUES
('Introduction to Programming', 'CS101', 3, 1),
('Data Structures', 'CS201', 4, 1),
('Calculus I', 'MATH101', 4, 2),
('Linear Algebra', 'MATH201', 3, 2),
('Physics I', 'PHYS101', 4, 3),
('Organic Chemistry', 'CHEM201', 4, 4),
('Database Systems', 'CS301', 3, 1);
-- Insert sample data into Registrations
INSERT INTO Registrations (student_id, course_id, registration_date, grade) VALUES
(1, 1, '2026-01-15', 'A'),
(1, 2, '2026-01-15', 'B+'),
(1, 3, '2026-01-15', 'A-'),
(2, 3, '2026-01-16', 'B'),
(2, 4, '2026-01-16', 'A'),
(3, 5, '2026-01-17', 'A'),
(4, 6, '2026-01-18', 'B+'),
(5, 1, '2026-01-19', 'A-'),
(5, 7, '2026-01-19', 'B'),
(2, 1, '2026-01-20', NULL),  -- Not graded yet
(3, 1, '2026-01-21', NULL);
-- Query 1: List all students enrolled in a specific course (replace ? with course_id, e.g., 1)
SELECT s.name AS student_name, s.email, r.registration_date, r.grade
FROM Registrations r
JOIN Students s ON r.student_id = s.student_id
WHERE r.course_id = ?
ORDER BY s.name;
-- Query 2: Find faculty members teaching more than 2 courses
SELECT f.name AS faculty_name, f.department, COUNT(c.course_id) AS courses_taught
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id, f.name, f.department
HAVING COUNT(c.course_id) > 2
ORDER BY courses_taught DESC;
-- Query 3: Get/Save courses with the highest number of registrations using GROUP BY
SELECT c.title AS course_title, c.code, COUNT(r.registration_id) AS enrollment_count
FROM Courses c
LEFT JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id, c.title, c.code
ORDER BY enrollment_count DESC
LIMIT 5;  -- Top 5 courses
-- Normalization Notes:
-- The schema is in 3NF:
-- - Students, Faculty, Courses are in 1NF/2NF with no partial dependencies.
-- - Registrations depends fully on its composite key (student_id, course_id), avoiding transitive dependencies.
-- - UNIQUE constraint on (student_id, course_id) prevents duplicate enrollments
-- Query Optimization Suggestions:
-- CREATE INDEX idx_registrations_student_id ON Registrations(student_id);
-- CREATE INDEX idx_registrations_course_id ON Registrations(course_id);
-- CREATE INDEX idx_courses_faculty_id ON Courses(faculty_id);
-- These indexes will speed up JOINs and GROUP BY operations, especially for large datasets.
```

Output:-

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 1 of 1

faculty_name	department	courses_taught
Dr. John Doe	Computer Science	3

SQL ▾ 1 / 1 1 - 5 of 5

course_title	code	enrollment_count
Introduction to Programming	CS101	4
Calculus I	MATH101	2
Organic Chemistry	CHEM201	1
Data Structures	CS201	1
Database Systems	CS301	1